Theory Question

Q38. What is the difference between map() and forEach()?

Return Value

- map(): Returns a new array with transformed elements.
- forEach(): Returns undefined; does not create a new array.

Purpose / Use Case

- map(): Use when you want to transform each element and get a new array.
- forEach(): Use when you want to perform side effects like logging, updating external variables, or DOM operations.

Mutation of Original Array

- Both do not mutate the original array unless the callback modifies it explicitly.

Chaining

- map(): Can be chained with other array methods because it returns a new array.
- forEach(): Cannot be chained directly because it returns undefined.

Callback Function

- Both take a callback function with arguments (element, index, array).

Typical Examples

- map():

  ```
  const numbers = [1, 2, 3];

  const squared = numbers.map(num => num * num);

  console.log(squared); // [1, 4, 9]
  ```

  forEach() Example

  ```
  const numbers = [1, 2, 3];

  numbers.forEach(num => console.log(num * 2));

  // Output: 2, 4, 6 (no array returned)
  ```

Q39. Difference between filter() and find()?

Purpose

- **filter(): Returns all elements that satisfy a condition.**
- **find(): Returns the first element that satisfies a condition.**

Return Value

- **filter(): Returns a new array (can be empty if no match).**
- **find(): Returns a single value (first matching element) or undefined if no match.**

Array Length of Result

- **filter(): Can return multiple elements, so length ≥ 0.**
- **find(): Returns only one element (or undefined).**

Use Case

- **filter(): When you need all matching elements.**
- **find(): When you need just the first match.**

Chaining

- **Both can be chained with other array methods.**

Example 1: filter()

```
const numbers = [1, 2, 3, 4, 5, 6];
const evenNumbers = numbers.filter(num => num % 2 === 0);
console.log(evenNumbers); // [2, 4, 6]
```

Example 2: find()

```
const numbers = [1, 2, 3, 4, 5, 6];
const firstEven = numbers.find(num => num % 2 === 0);
console.log(firstEven); // 2
```

Q40. What does reduce() do?

- The reduce() method **reduces an array to a single value** by executing a **callback function on each element**.
- The callback **accumulates a result** as it iterates through the array.

Syntax

```
arr.reduce(callback(accumulator, currentValue, index, array), initialValue)
```

- **callback** – Function executed on each element.
  - accumulator – Holds accumulated value from previous iteration.
  - currentValue – Current array element.
  - index – Index of current element (optional).
  - array – Original array (optional).
- **initialValue** – Optional starting value for accumulator. If not provided, first element of array is used.

Example with source code

```
const numbers = [1, 2, 3, 4, 5];
const sum = numbers.reduce((acc, curr) => acc + curr, 0);
console.log(sum); // 15
```

Q41. How do you check if a variable is an array?

1. Using Array.isArray() (Recommended)

```
const arr = [1, 2, 3];
console.log(Array.isArray(arr)); // true
```

```
const num = 10;
console.log(Array.isArray(num)); // false
```

- Returns true if the variable is an array.
- Works for arrays from different frames/windows, unlike instanceof.

---

2. Using instanceof Operator

```
const arr = [1, 2, 3];
console.log(arr instanceof Array); // true

const obj = {};
console.log(obj instanceof Array); // false
```

- Checks if the object inherits from Array.prototype.
- Limitation: Can fail across iframes or windows.

---

3. Using Object.prototype.toString (Legacy/Advanced)

```
const arr = [1, 2, 3];
console.log(Object.prototype.toString.call(arr)); // "[object Array]"

const obj = {};
console.log(Object.prototype.toString.call(obj)); // "[object Object]"
```

- Reliable for cross-frame arrays.
- Less commonly used now due to Array.isArray().

Q42. What is array destructuring?
Array destructuring is a JavaScript feature that allows you to extract values from an array and assign them to variables in a single line.
Introduced in ES6, it makes code more concise and readable.

Syntax

```
const [var1, var2, var3] = array;
```

- var1 → assigned the first element of the array
- var2 → assigned the second element, and so on

Example 1: Basic Destructuring

```
const fruits = ["Apple", "Banana", "Mango"];
const [first, second, third] = fruits;

console.log(first);  // "Apple"
```

```
console.log(second); // "Banana"
console.log(third);  // "Mango"
```

Example 2: Skipping Elements

```
const numbers = [10, 20, 30, 40];
const [, second, , fourth] = numbers;

console.log(second); // 20
console.log(fourth); // 40
```

- Use **commas** to skip elements.

Example 3: Default Values

```
const numbers = [1];
const [a = 10, b = 20] = numbers;

console.log(a); // 1
console.log(b); // 20 (default used)
```

Example 4: Swapping Variables

```
let x = 5, y = 10;
[x, y] = [y, x];

console.log(x); // 10
console.log(y); // 5
```

Q43. What is the spread operator in arrays?

- The spread operator (...) allows an iterable (like an array) to be expanded into individual elements.
- Introduced in ES6, it is used to copy, merge, or pass array elements easily.

Key Uses in Arrays

1. Copying an Array

```
const arr1 = [1, 2, 3];
const arr2 = [...arr1]; // shallow copy

console.log(arr2); // [1, 2, 3]
console.log(arr1 === arr2); // false (different references)
```

---

2. Merging Arrays

```
const arr1 = [1, 2];
const arr2 = [3, 4];
const merged = [...arr1, ...arr2];

console.log(merged); // [1, 2, 3, 4]
```

Q44.What is the time complexity of array operations?

Time Complexity of Array Operations

Assume a JavaScript array implemented as a dynamic array (like in V8 engine).

Operation	Time Complexity	Notes
Access by index (arr[i])	O(1)	Direct access via numeric index
Update by index (arr[i] = x)	O(1)	Direct assignment
Push (arr.push(x))	O(1) amortized	Usually O(1), occasionally O(n) when resizing the array
Pop (arr.pop())	O(1)	Removes last element
Shift (arr.shift())	O(n)	Removes first element; all elements must shift left
Unshift (arr.unshift(x))	O(n)	Adds element at start; all elements must shift right
Insert/Delete in middle	O(n)	Elements must be shifted
Search (linear) (indexOf, includes)	O(n)	Checks each element until found
Iteration (for, forEach, map)	O(n)	Visits each element once

Concat / Slice	**O(n)**	**Creates new array copying elements**
Sort (arr.sort())	**O(n log n)**	**Depends on JS engine; typically QuickSort or TimSort**

Q45. How do you make arrays immutable?

In JavaScript, arrays are mutable by default, meaning their elements can be changed. To make arrays immutable, we prevent direct modification and create new arrays for changes.

1. Using const

```
const arr = [1, 2, 3];
arr.push(4); // Allowed! Mutates array
console.log(arr); // [1, 2, 3, 4]
```

- const only prevents reassignment, not mutation.
- To make it truly immutable, we need methods that return new arrays.


```
const arr = [1, 2, 3];
arr.push(4); // Allowed! Mutates array
console.log(arr); // [1, 2, 3, 4]
```

- const only prevents reassignment, not mutation.
- To make it truly immutable, we need methods that return new arrays.

Q46. What is a string in JavaScript?

A string is a sequence of characters used to represent text.
In JavaScript, strings are immutable, meaning once created, their contents cannot be changed.
Strings can contain letters, numbers, symbols, whitespace, or special characters.

Example

```
// Using double quotes
const str1 = "Hello";

// Using single quotes
const str2 = 'World';

// Using template literals (backticks) - ES6
const name = "Alice";
const str3 = `Hello, ${name}!`; // "Hello, Alice!"
```

Q47. Are strings mutable or immutable?

- Strings in JavaScript are immutable.
- Once a string is created, its characters cannot be changed individually.

- Any operation that seems to “change” a string actually creates a new string.

1. Example: Attempting to Change a Character

```
let str = "Hello";

str[0] = "h"; // ✕ Attempt to change first character

console.log(str); // "Hello" (unchanged)
```

- Even though we tried to change the first character, the original string **remains the same**.

2. Modifying Strings Creates New Strings

```
let str = "Hello";

let newStr = str.replace("H", "h");

console.log(str);     // "Hello" (original unchanged)

console.log(newStr);  // "hello" (new string created)
```

Q48. Difference between substring(), slice(), and substr()?

Difference Between substring(), slice(), and substr()

Feature / Method	**substring(start, end)**	**slice(start, end)**	**substr(start, length)**
Purpose	**Extracts a part of the string**	**Extracts a part of the string**	**Extracts a part of the string**
Parameters	**start index, end index (not included)**	**start index, end index (not included)**	**start index, length of substring**
Negative Values	**Treated as 0**	**Counts from end if negative**	**Counts from end if negative**
Mutates Original String	**✕ No**	**✕ No**	**✕ No**
Return Value	**New string**	**New string**	**New string**

Use Case	**Extract substring using indexes**	**Extract substring using indexes, supports negative indexing**	**Extract substring using start + length**
Deprecation	**Safe to use**	**Safe to use**	**Legacy – discouraged in modern JS**

Q49. How do you convert string to array?

There are several ways to convert a string into an array depending on your requirements.

---

1. Using split() (Most Common)

```
const str = "JavaScript,Python,Java";
const arr = str.split(","); // Split by comma

console.log(arr); // ["JavaScript", "Python", "Java"]
```

- **split(separator, limit) divides the string into substrings.**
  - **separator → defines where to split (comma, space, character, or regex).**
  - **limit → optional, limits number of array elements.**

```
const str = "Hello World";
const arr = str.split(""); // Split into individual characters

console.log(arr); // ["H", "e", "l", "l", "o", " ", "W", "o", "r", "l", "d"]
```

---

2. Using Spread Operator (...)

```
const str = "Hello";
const arr = [...str];

console.log(arr); // ["H", "e", "l", "l", "o"]
```

- **Expands string into individual characters.**
- **Very concise and modern (ES6+).**

---

3. Using Array.from()

```
const str = "Hello";
const arr = Array.from(str);

console.log(arr); // ["H", "e", "l", "l", "o"]
```

- **Converts any iterable (like strings) into an array.**

- **Can also apply a mapping function while converting:**

```
const arr = Array.from("Hello", char => char.toUpperCase());
console.log(arr); // ["H", "E", "L", "L", "O"]
```

Q50. How do you convert an array to string?

There are several ways to convert an array into a string depending on the format you want.

---

1. Using toString()

```
const arr = [1, 2, 3, 4];
const str = arr.toString();

console.log(str); // "1,2,3,4"
```

- **Converts the array elements to a string separated by commas.**
- **Does not allow custom separators.**

---

2. Using join()

```
const arr = ["JavaScript", "Python", "Java"];
const str = arr.join(", "); // Specify separator

console.log(str); // "JavaScript, Python, Java"
```

- **join(separator) lets you choose the separator (comma, space, hyphen, etc.).**
- **Returns a new string, original array unchanged.**

```
const arr = [1, 2, 3];
console.log(arr.join(" - ")); // "1 - 2 - 3"
```

---

3. Using Template Literals (for Simple Concatenation)

```
const arr = ["Hello", "World"];
const str = `${arr[0]} ${arr[1]}`;

console.log(str); // "Hello World"
```

- **Useful for specific formatting, but less flexible for large arrays.**

Q51. What is template literal?

- **Template literals are a way to define strings in JavaScript using backticks (`) instead of quotes.**
- **Introduced in ES6, they allow embedded expressions, multi-line strings, and string interpolation.**

---

1. Basic Syntax

```
const name = "Alice";
```

const greeting = `Hello, ${name}!`;

console.log(greeting); // "Hello, Alice!"

- **${expression} allows you to embed variables or expressions inside strings.**

---

2. Multi-line Strings
const message = `This is a
multi-line
string.`;

console.log(message);
// Output:
// This is a
// multi-line
// string.

- **No need for \n or string concatenation.**

---

3. Expressions Inside Template Literals
const a = 5;
const b = 10;
console.log(`Sum of ${a} + ${b} = ${a + b}`); // "Sum of 5 + 10 = 15"

- **Can include any valid JavaScript expression inside ${}.**

Q52. What is string interpolation?

- String interpolation is the process of inserting variables or expressions directly into a string.
- In JavaScript, it is done using template literals (backticks `) with ${} placeholders.
- Makes strings dynamic and readable compared to traditional concatenation.

---

1. Example Using Template Literals
const name = "Alice";
const age = 25;

const message = `My name is ${name} and I am ${age} years old.`;
console.log(message);
// Output: "My name is Alice and I am 25 years old."

- ${name} → replaced with the value of name.
- ${age} → replaced with the value of age.

---

2. Example with Expressions

```
const a = 5;
const b = 10;

console.log(`Sum of ${a} + ${b} = ${a + b}`);
// Output: "Sum of 5 + 10 = 15"
```

- Any JavaScript expression can be evaluated inside ${}.

---

3. Example vs Traditional Concatenation

Without Interpolation:

```
const name = "Alice";
const greeting = "Hello, " + name + "!";
console.log(greeting); // "Hello, Alice!"
```

With Interpolation:

```
const greeting = `Hello, ${name}!`;
console.log(greeting); // "Hello, Alice!"
```

- Interpolation is cleaner, more readable, and less error-prone.

Q53. What is charAt()?

- The charAt() method is a string method that returns the character at a specified index in a string.
- Indexing starts at 0, meaning the first character has index 0.
- If the index is out of range, it returns an empty string ("").

---

1. Syntax

```
str.charAt(index)
```

- index → the position of the character you want to retrieve.

---

2. Examples

```
const str = "JavaScript";

console.log(str.charAt(0));  // "J" (first character)
console.log(str.charAt(4));  // "S"
console.log(str.charAt(20)); // "" (index out of range)
```

---

3. Comparison with Bracket Notation

```
console.log(str[0]); // "J"
console.log(str[20]); // undefined
```

- charAt() returns empty string for out-of-range indexes.

- Bracket notation (str[index]) returns undefined if index is out of range.

Q54. What is the includes() method?
The includes() method is a string or array method that checks if a given value exists within the string or array.
Returns a boolean: true if found, false otherwise.

1. Syntax

For Strings:
str.includes(searchString, position)

- searchString → the substring to search for
- position → optional, the index to start searching from (default: 0)

For Arrays:
arr.includes(valueToFind, fromIndex)

- valueToFind → element to search
- fromIndex → optional, start index for search

---

2. Examples

String Example:
```
const str = "JavaScript is fun";

console.log(str.includes("Script"));  // true
console.log(str.includes("script"));  // false (case-sensitive)
console.log(str.includes("fun", 5));  // true (start searching from index 5)
```

Array Example:
```
const arr = [1, 2, 3, 4, 5];

console.log(arr.includes(3));      // true
console.log(arr.includes(6));      // false
console.log(arr.includes(2, 2));   // false (search starts from index 2)
```

Q55. What is indexOf()?

The indexOf() method is commonly used in programming languages like **JavaScript** and **Java** to find the position (index) of a specified element or substring in a string or array.

Here's a breakdown:

---

1. In JavaScript (for strings and arrays)

Syntax for strings:

string.indexOf(searchValue, fromIndex)

- searchValue → the substring you want to find.
- fromIndex (optional) → the position in the string to start searching. Default is 0.
- **Returns** → the index of the first occurrence of the substring, or -1 if not found.

Example:
```
let str = "Hello World";
console.log(str.indexOf("World")); // Output: 6
console.log(str.indexOf("world")); // Output: -1 (case-sensitive)
```

Syntax for arrays:
array.indexOf(searchElement, fromIndex)

- Works similarly, returns the index of the first occurrence, or -1 if not found.

Example:
```
let arr = [10, 20, 30, 20];
console.log(arr.indexOf(20)); // Output: 1
console.log(arr.indexOf(40)); // Output: -1
```

---

2. In Java (for strings)

Syntax:
```
int index = string.indexOf(String str)
int index = string.indexOf(String str, int fromIndex)
```

- Returns the index of the first occurrence of str in the string.
- Returns -1 if the substring is not found.

Example:
```
String str = "Hello World";
System.out.println(str.indexOf("World")); // Output: 6
System.out.println(str.indexOf("world")); // Output: -1
```

Q56. What is asynchronous programming?

Asynchronous programming is a programming paradigm that allows your code to start a task and move on to other tasks before the first one finishes, instead of waiting (blocking) for it to complete. This is especially useful for operations that take time, like reading files, making network requests, or querying a database.

◆ **Key Idea**

- Synchronous (blocking): Tasks run one after another, each must finish before the next starts.

- Asynchronous (non-blocking): Tasks can start and continue running in the background while the program handles other tasks.

Example in JavaScript

Synchronous code:
```
console.log("Start");
console.log("Task 1");
console.log("End");
```

Output:
Start
Task 1
End

Asynchronous code using setTimeout:
```
console.log("Start");

setTimeout(() => {
console.log("Task 1 done");
}, 2000); // waits 2 seconds

console.log("End");
```

Output:
Start
End
Task 1 done

Notice how "End" is printed before "Task 1 done"—the program didn't wait for the 2-second delay.

🔹 Common Use Cases

- API calls / network requests
- File reading/writing
- Timers or delays
- Database queries

🔹 How it works

Asynchronous programming often uses:

- Callbacks – Functions passed as arguments to execute later.
- Promises – Objects representing a future value.
- async/await – Syntactic sugar to write asynchronous code more like synchronous code.

Example with Promises:

```
fetch("https://api.example.com/data")
.then(response => response.json())
.then(data => console.log(data))
.catch(error => console.error(error));
```

Here, fetch is asynchronous, and the program continues running while waiting for the network response.

Q57. What is the event loop?

The event loop is a core concept in asynchronous programming (especially in JavaScript) that allows the program to handle multiple tasks without blocking. It continuously checks for tasks to execute and ensures that asynchronous operations run in the right order.

Key Idea

- JavaScript is single-threaded, meaning it can execute one task at a time.
- The event loop allows JavaScript to perform non-blocking operations (like I/O, timers, network requests) while still running other code.

🔹 How It Works

1. Call Stack
   - Where your code executes line by line.
   - Example: function calls go here.
2. Web APIs / Background Tasks
   - Browser or Node.js handles things like setTimeout, network requests, DOM events.
   - These run outside the main thread.
3. Callback Queue (or Task Queue)
   - When an asynchronous operation is done, its callback is placed here.
4. Event Loop
   - Continuously checks the call stack.
   - If the call stack is empty, it takes the first task from the callback queue and pushes it to the call stack to execute.

🔹 Example

```
console.log("Start");

setTimeout(() => {
console.log("Inside Timeout");
}, 0);

console.log("End");
```

Output:
Start
End
Inside Timeout

Explanation:

1. "Start" is logged → added to call stack and executed.
2. setTimeout is called → callback sent to Web API → after 0ms, goes to callback queue.
3. "End" is logged → call stack empty.
4. Event loop takes setTimeout callback from the queue → executed → logs "Inside Timeout".

Q58. What are microtasks and macrotasks?

In JavaScript's asynchronous programming model, tasks are handled in two main categories by the event loop: macrotasks and microtasks. Understanding the difference explains why some async code executes before other code even if they were scheduled later.

🔹 1. Macrotasks (Tasks)

- Also called just tasks.
- Examples:
  - setTimeout, setInterval
  - setImmediate (Node.js)
  - I/O events, UI rendering, network requests
- Scheduled to run in the macrotask queue.
- The event loop picks one macrotask at a time, runs it completely, then moves to the next.

🔹 2. Microtasks

- Higher priority than macrotasks.
- Examples:
  - Promises (.then, .catch, .finally)
  - process.nextTick() (Node.js)
  - MutationObserver (browser)
- Placed in the microtask queue, which is executed immediately after the current task finishes, before the next macrotask.

🔹 How They Work Together

1. Event loop executes a macrotask (e.g., main code or setTimeout callback).
2. After this macrotask finishes, the event loop runs all microtasks in the microtask queue.

3. Only then the event loop picks the next macrotask.

Q59. What is a Promise?

A Promise in JavaScript is an object that represents the eventual result of an asynchronous operation. It's a way to handle asynchronous tasks more cleanly than callbacks, avoiding "callback hell."

🔹 Key Idea

A Promise is like a promise in real life:

> "I promise to give you a value later—either I succeed or I fail."

- Pending – Initial state, operation hasn't completed yet.
- Fulfilled (Resolved) – Operation completed successfully.
- Rejected – Operation failed.

🔹 Syntax

```
let promise = new Promise((resolve, reject) => {
// asynchronous operation
let success = true;

if (success) {
resolve("Task completed");   // fulfilled
} else {
reject("Task failed");       // rejected
}
});
```

🔹 Using Promises

With .then() and .catch()

```
promise
.then((result) => {
console.log("Success:", result);
})
.catch((error) => {
console.log("Error:", error);
});
```

- .then() → runs when the Promise is fulfilled.
- .catch() → runs when the Promise is rejected.

🔹 Example with Real Async Task

```
function fetchData() {
return new Promise((resolve, reject) => {
setTimeout(() => {
let data = { name: "Alice" };
resolve(data); // succeed
// reject("Error fetching data"); // fail
}, 2000);
});
}

fetchData()
.then((data) => console.log("Data received:", data))
.catch((err) => console.log(err));
```

Output after 2 seconds:
Data received: { name: "Alice" }

🔹 Advantages of Promises

1. Avoid callback hell – cleaner syntax for async tasks.
2. Chainable – you can chain .then() for multiple async operations.
3. Error handling – use .catch() to handle errors in one place.
4. Works with async/await for even more readable code.

Q60. States of a Promise?

In JavaScript, a Promise is an object representing the eventual completion or failure of an asynchronous operation. A Promise can exist in three states:

1. Pending
   - Initial state when the Promise is created.
   - The operation hasn't completed yet, and neither resolve nor reject has been called.
2. Fulfilled (Resolved)
   - The operation completed successfully.
   - The then() handler is called with the resolved value.
3. Rejected
   - The operation failed.
   - The catch() handler is called with the reason (error).

Quick Example:

```
let promise = new Promise((resolve, reject) => {
let success = true;
if (success) {
resolve("Task completed");  // Fulfilled
} else {
```

```
reject("Task failed");      // Rejected
}
});

promise
.then((result) => console.log(result))  // called if fulfilled
.catch((error) => console.log(error)); // called if rejected
```

Q61. Difference between Promise and callback?

---

1. Definition

- Callback: A function passed as an argument to another function to be executed after an asynchronous operation completes.
- Promise: An object representing the eventual result (success or failure) of an asynchronous operation.

---

2. Syntax & Readability

- Callback: Can lead to “callback hell” if nested deeply.
- Promise: Allows chaining with .then() and .catch(), making code more readable.

---

3. Error Handling

- Callback: Errors must be handled manually inside the callback.
- Promise: Has built-in error handling via .catch().

---

4. Return Values

- Callback: Cannot return a value directly; must pass via callback.
- Promise: Can return a value that can be used in subsequent .then().

---

5. Execution

- Callback: Executes only when explicitly called.
- Promise: Executes immediately when created; .then() just registers handlers.

Callback version:

```
function getData(callback) {
setTimeout(() => {
callback(null, "Data received");
}, 1000);
}
```

```
getData((err, data) => {
if (err) console.log(err);
else console.log(data);
});
```

Promise version:

```
let getData = new Promise((resolve, reject) => {
setTimeout(() => {
resolve("Data received");
}, 1000);
});

getData
.then((data) => console.log(data))
.catch((err) => console.log(err));
```

Q62. What is callback hell?

Callback Hell

Definition:
Callback hell occurs when you have multiple nested callbacks, making the code **hard to read, maintain, and debug**. It is also called **"Pyramid of Doom"** because nested callbacks create a pyramid-like indentation.

Why it happens:

- Asynchronous operations (like API calls, file reading, timers) are nested inside each other.
- Each callback depends on the previous one, causing deep nesting.

Example of Callback Hell:

```
doSomething(function(result1) {
doSomethingElse(result1, function(result2) {
doAnotherThing(result2, function(result3) {
doFinalTask(result3, function(result4) {
console.log("All tasks done");
});
});
});
});
```

- Notice the **deep indentation**—this is the "Pyramid of Doom."
- Hard to read and harder to handle errors.

How to Avoid It:

1. **Use Promises** – .then() chaining makes it cleaner.
2. **Use Async/Await** – makes asynchronous code look **synchronous**.
3. **Modularize Functions** – break tasks into smaller functions.

Promise Version (Cleaner):

```
doSomething()
.then(result1 => doSomethingElse(result1))
.then(result2 => doAnotherThing(result2))
.then(result3 => doFinalTask(result3))
.then(final => console.log("All tasks done"))
.catch(err => console.error(err));
```

Q63. What is async/await?

Async/Await

Definition:
async/await is a modern way to handle asynchronous operations in JavaScript. It allows you to write asynchronous code like synchronous code, making it easier to read and maintain.

How it works:

1. async function
   - A function declared with async always returns a Promise.
   - Inside it, you can use await.
2. await keyword
   - Pauses the execution of the async function until the Promise is resolved or rejected.
   - Can only be used inside an async function.

Example:

```
function fetchData() {
return new Promise((resolve, reject) => {
setTimeout(() => {
resolve("Data received");
}, 1000);
});
}

async function getData() {
const result = await fetchData(); // waits here until promise resolves
console.log(result);
}
```

getData(); // Output after 1 second: "Data received"

Benefits of async/await

1. Makes asynchronous code look synchronous → easier to read.
2. Avoids callback hell.
3. Easy error handling with try/catch.

Q64. Error handling in async/await?

Error Handling in Async/Await

When using async/await, errors in asynchronous code can be caught just like synchronous code using try...catch blocks.

Syntax Example

```
async function fetchData() {
try {
let response = await fetch("https://api.example.com/data");
let data = await response.json();
console.log(data);
} catch (error) {
console.error("Error occurred:", error);
}
}

fetchData();
```

Explanation:

- The await pauses execution until the Promise resolves.
- If the Promise rejects or throws an error, it is caught in the catch block.
- This avoids using .catch() chaining like with Promises.

Q65. What is Promise chaining?

Definition:
Promise chaining is a technique where you perform multiple asynchronous operations sequentially by returning a new Promise from the .then() handler. This allows one operation to start after the previous one has completed, keeping the code clean and readable.

How it works

1. Each .then() can return a value or a new Promise.
2. The next .then() receives the resolved value from the previous .then().

3. Errors can be handled at the end using .catch().

Example

```
function getNumber() {
return new Promise((resolve, reject) => {
setTimeout(() => resolve(5), 1000);
});
}

getNumber()
.then(result => {
console.log(result); // 5
return result * 2;    // pass to next then
})
.then(result => {
console.log(result); // 10
return result + 3;    // pass to next then
})
.then(result => {
console.log(result); // 13
})
.catch(error => console.error(error));
```

Q66. What is Promise.all()?

Promise.all()

Definition:
Promise.all() is a method that takes an array (or iterable) of Promises and returns a single Promise that:

1. Resolves when all the input Promises resolve, with an array of their results.
2. Rejects immediately if any one of the input Promises rejects, with the reason of the first rejection.

Example

```
const p1 = Promise.resolve(10);
const p2 = Promise.resolve(20);
const p3 = Promise.resolve(30);

Promise.all([p1, p2, p3])
.then(results => {
console.log(results); // [10, 20, 30]
})
.catch(error => console.error(error));
```

Example with Rejection:

```
const p1 = Promise.resolve(10);
const p2 = Promise.reject("Error occurred");
const p3 = Promise.resolve(30);

Promise.all([p1, p2, p3])
.then(results => console.log(results))
.catch(error => console.error(error));
// Output: "Error occurred"
```

Q67. What is Promise.race()?

Definition:
Promise.race() is a method that takes an array (or iterable) of Promises and returns a single Promise that settles as soon as the first Promise in the array settles, whether it is fulfilled or rejected.

Key Points

1. Resolves or rejects with the value or reason of the first settled Promise.
2. Useful when you want the fastest response among multiple asynchronous operations.

Example: Fastest Promise Wins

```
const p1 = new Promise(resolve => setTimeout(() => resolve("First"), 1000));
const p2 = new Promise(resolve => setTimeout(() => resolve("Second"), 2000));
const p3 = new Promise((_, reject) => setTimeout(() => reject("Error!"), 500));

Promise.race([p1, p2, p3])
.then(result => console.log("Resolved:", result))
.catch(error => console.error("Rejected:", error));

// Output after 500ms: "Rejected: Error!"
```

Explanation:

- Even though p1 and p2 might resolve later, p3 rejects first, so Promise.race() immediately rejects with "Error!".

Q68. What is Promise.allSettled()?

Promise.allSettled()

Definition:
Promise.allSettled() is a method that takes an array (or iterable) of Promises and returns a single Promise that resolves after all the input Promises have settled, regardless of whether they were fulfilled or rejected.

- It never rejects.
- Provides detailed results for each Promise, including status and value/reason.

Result Format

The resolved array contains objects like:
{ status: "fulfilled", value: ... }
{ status: "rejected", reason: ... }

Example
```
const p1 = Promise.resolve(10);
const p2 = Promise.reject("Error occurred");
const p3 = Promise.resolve(30);

Promise.allSettled([p1, p2, p3])
.then(results => console.log(results));
```

Output:
```
[
{ status: "fulfilled", value: 10 },
{ status: "rejected", reason: "Error occurred" },
{ status: "fulfilled", value: 30 }
]
```

Explanation:

- All promises are waited for, even if some reject.
- Each result tells whether the Promise fulfilled or rejected, along with its value or reason.

Q69. What is setTimeout() vs setInterval()?

1. setTimeout()

Definition:

- setTimeout() executes a function once after a specified delay (in milliseconds).

Syntax:
```
setTimeout(function, delay, param1, param2, ...);
```

Example:
```
setTimeout(() => {
console.log("Executed after 2 seconds");
}, 2000);
```

Explanation:

- The function runs once after 2000ms (2 seconds).

---

2. setInterval()

Definition:

- setInterval() executes a function repeatedly at fixed intervals (in milliseconds) until cleared.

Syntax:
setInterval(function, interval, param1, param2, ...);

Example:
```
let count = 0;
const intervalId = setInterval(() => {
count++;
console.log("Count:", count);
if(count === 5) clearInterval(intervalId); // stop after 5 times
}, 1000);
```

Explanation:

- Function runs every 1000ms (1 second) until clearInterval() is called.

Q70. How to avoid callback hell?

Callback Hell

- Definition: Occurs when multiple nested callbacks make code hard to read, maintain, or debug.
- Also called “Pyramid of Doom.”

How to Avoid Callback Hell

1. Use Promises

    - Promises allow chaining with .then() instead of nesting callbacks.

```
doSomething()
 .then(result => doSomethingElse(result))
 .then(result => doAnotherThing(result))
 .then(final => console.log(final))
 .catch(err => console.error(err));
```

2. **Use Async/Await**
- Write asynchronous code that **looks synchronous**.
```javascript id="aa1ch"
async function processTasks() {
```

```
try {
const result1 = await doSomething();
const result2 = await doSomethingElse(result1);
const final = await doAnotherThing(result2);
console.log(final);
} catch(err) {
console.error(err);
}
}
processTasks();
```

3. Modularize Functions
   - Break tasks into smaller, reusable functions instead of nesting everything.
4. Error Handling
   - Centralized error handling using .catch() (Promises) or try/catch (async/await).

Q71. What is the fetch API?

Fetch API

Definition:
The Fetch API is a modern JavaScript interface used to make HTTP requests (like GET, POST, PUT, DELETE) to servers. It returns a Promise that resolves to the response of the request.

Key Features

1. Uses Promises (no callbacks).
2. Cleaner and more powerful than older methods like XMLHttpRequest.
3. Supports async/await for better readability.
4. Works for API calls, fetching data, sending data to servers.

Basic Syntax

```
fetch(url, options)
.then(response => response.json())
.then(data => console.log(data))
.catch(error => console.error(error));
```

Example (GET Request)

```
fetch("https://jsonplaceholder.typicode.com/users")
.then(response => response.json())
.then(data => console.log(data))
.catch(error => console.error("Error:", error));
```

Example using async/await

```
async function getData() {
try {
const response = await fetch("https://jsonplaceholder.typicode.com/users");
const data = await response.json();
console.log(data);
} catch (error) {
console.error("Error:", error);
}
}

getData();
```

Q72. How do you handle API errors?

Handling API Errors

When working with APIs (e.g., using fetch), errors can occur in two main ways:

1. Network errors (no internet, server down)
2. HTTP errors (like 404, 500)

---

1. Using .then() and .catch()

```
fetch("https://api.example.com/data")
.then(response => {
if (!response.ok) {
throw new Error("HTTP error: " + response.status);
}
return response.json();
})
.then(data => console.log(data))
.catch(error => console.error("Error:", error));
```

Explanation:

- response.ok checks if status is 200–299
- If not, we manually throw an error
- .catch() handles all errors

---

2. Using async/await with try-catch

```
async function getData() {
try {
const response = await fetch("https://api.example.com/data");

if (!response.ok) {
throw new Error("HTTP error: " + response.status);
```

```
}

const data = await response.json();
console.log(data);
} catch (error) {
console.error("Error:", error);
}
}

getData();
```

Q73. What is debouncing?

Definition:
Debouncing is a technique used to limit how often a function is executed. It ensures that a function is only called after a certain delay has passed since the last time it was triggered.

Why it is used

- Prevents unnecessary repeated function calls
- Improves performance
- Common in search input, resize events, scroll events

Example Scenario

Search bar typing:

- User types: a → ap → app → appl → apple
- Without debouncing → API called 5 times
- With debouncing → API called only once (after user stops typing)

Basic Example

```
function debounce(func, delay) {
let timer;
return function(...args) {
clearTimeout(timer);
timer = setTimeout(() => {
func.apply(this, args);
}, delay);
};
}

// Usage
const search = debounce(() => {
console.log("API call");
```

```
}, 500);
```

How it works

1. Function is called → timer starts
2. If called again before delay → previous timer is cleared
3. Only the last call executes after delay

Q74. What is throttling?

Throttling

Definition:
Throttling is a technique used to limit how often a function executes by ensuring it runs at most once in a specified time interval, no matter how many times it is triggered.

Why it is used

- Controls high-frequency events
- Improves performance
- Prevents excessive function calls

Q75. Difference between synchronous and asynchronous code?

Synchronous Code

Definition:
Synchronous code executes **line by line**, meaning each operation waits for the previous one to complete before executing.

Example:
```
console.log("Start");
console.log("Middle");
console.log("End");
```

Output:
Start
Middle
End

Key Point:

- Blocking in nature (next task waits)

Asynchronous Code

Definition:
Asynchronous code allows tasks to run **independently**, so the program does not wait for one task to finish before moving to the next.

Example:
```
console.log("Start");

setTimeout(() => {
console.log("Middle");
}, 2000);

console.log("End");
```

Output:
Start
End
Middle

MCQ Question

Q1. What will be the output?
```
const arr = [1, 2, 3];
const result = arr.map(x => x * 2);
console.log(result);
```

A. [1, 2, 3]
B. [2, 4, 6]
C. undefined
D. Error

Answer: B

Explanation:
map() returns a new array by applying the function to each element.
Here each element is multiplied by 2.
Original array remains unchanged.
So result becomes [2, 4, 6].

Q2. What will forEach return?
```
const arr = [1, 2, 3];
const result = arr.forEach(x => x * 2);
console.log(result);
```

A. [2, 4, 6]
B. undefined
C. [1, 2, 3]
D. Error

Answer: B

Explanation:
forEach() does not return a value.
It is used for side effects like logging or updating variables.
Even though multiplication happens, result is not stored.
Hence it returns undefined.

Q3. Output of filter
[1,2,3,4].filter(x => x > 2)

A. [1,2]
B. [3,4]
C. 3
D. undefined

Answer: B

Explanation:
filter() returns all elements satisfying the condition.
Here condition is x > 2.
So elements 3 and 4 are returned.
Result is a new array [3,4].

Q4. Output of find
[1,2,3,4].find(x => x > 2)

A. [3,4]
B. 3
C. 4
D. undefined

Answer: B

Explanation:
find() returns only the first matching element.
Condition is x > 2.
First matching element is 3.
So output is 3.

Q5. reduce output
[1,2,3].reduce((a,b)=>a+b,0)

A. 6
B. 5
C. 3
D. Error

Answer: A

Explanation:
reduce() accumulates values.
Initial value is 0.
0+1=1 → 1+2=3 → 3+3=6.
Final result is 6.

Q6. Array check
Array.isArray([1,2,3])

A. true
B. false
C. undefined
D. Error

Answer: A

Explanation:
Array.isArray() checks if value is an array.
Here input is an array.
So it returns true.
Recommended method for array checking.

Q7. Spread operator output
```
const arr = [1,2];
const newArr = [...arr, 3];
console.log(newArr);
```

A. [1,2]
B. [1,2,3]
C. [3,1,2]
D. Error

Answer: B

Explanation:
Spread operator expands array elements.
It copies elements of arr into new array.
Then 3 is added.
Result is [1,2,3].

Q8. Time complexity of push

A. O(n)
B. O(1)
C. O(log n)
D. O(n^2)

Answer: B

Explanation:
push() adds element at end.
No shifting required.
Hence constant time complexity.
Occasionally resizing may occur but amortized O(1).

Q9. String immutability
```
let str = "Hello";
str[0] = "h";
console.log(str);
```

A. hello
B. Hello
C. h
D. Error

Answer: B

Explanation:
Strings are immutable in JavaScript.
You cannot change characters directly.
Assignment is ignored.
So string remains "Hello".

Q10. split output
```
"abc".split("")
```

A. ["abc"]
B. ["a","b","c"]

C. "abc"
D. Error

Answer: B

Explanation:
split("") breaks string into characters.
Each character becomes an array element.
So output is ["a","b","c"].
Useful for iteration.

Q11. join output
[1,2,3].join("-")

A. "1-2-3"
B. [1-2-3]
C. "123"
D. Error

Answer: A

Explanation:
join() converts array to string.
Separator "-" is used.
Elements are combined with dash.
Output is "1-2-3".

Q12. Template literal
let name = "A";
console.log(`Hi ${name}`);

A. Hi A
B. Hi ${name}
C. Error
D. undefined

Answer: A

Explanation:
Template literals use backticks.
${} inserts variable value.
Here name is "A".
So output is "Hi A".

Q13. charAt
"Hello".charAt(1)

A. H
B. e
C. l
D. undefined

Answer: B

Explanation:
Index starts from 0.
Index 1 corresponds to 'e'.
charAt() returns character at index.
So result is 'e'.

Q14. includes
"JavaScript".includes("Script")

A. true
B. false
C. undefined
D. Error

Answer: A

Explanation:
includes() checks substring existence.
"Script" exists in string.
Case-sensitive match.
So returns true.

Q15. indexOf
"Hello".indexOf("o")

A. 4
B. 3
C. 5
D. -1

Answer: A

Explanation:
Indexing starts at 0.
'o' is at position 4.

indexOf() returns first occurrence.
So result is 4.

Q16. Promise state
new Promise(()=>{})

A. fulfilled
B. rejected
C. pending
D. error

Answer: C

Explanation:
No resolve or reject is called.
Promise remains in initial state.
That state is called pending.
So output is pending.

Q17. setTimeout output
console.log("A");
setTimeout(()=>console.log("B"),0);
console.log("C");

A. A B C
B. A C B
C. B A C
D. C A B

Answer: B

Explanation:
JS is asynchronous.
setTimeout goes to callback queue.
"A" and "C" execute first.
Then "B" prints.

Q18. Promise.all

If one promise fails:
A. all succeed
B. returns partial result
C. rejects immediately
D. ignores error

Answer: C

Explanation:
Promise.all() waits for all promises.
But if any one rejects, it fails immediately.
Remaining results are ignored.
Hence rejects instantly.

Q19. Debounce behavior

A. Runs continuously
B. Runs after delay once
C. Runs immediately always
D. Never runs

Answer: B

Explanation:
Debouncing delays execution.
It runs only after user stops triggering.
Prevents repeated calls.
Common in search inputs.

Q20. Throttle behavior

A. Runs once after delay
B. Runs repeatedly at interval
C. Never runs
D. Runs randomly

Answer: B

Explanation:
Throttling limits execution rate.
Function runs at fixed intervals.
Even if triggered many times.
Useful for scroll events.

Q21. What is async programming?

A. Blocking execution
B. Non-blocking execution

C. No execution
D. Sequential only

Answer: B

Explanation:
Async programming allows tasks to run independently.
Code does not wait for completion.
Improves performance.
Used in APIs and timers.

Q22. Event loop role

A. Executes sync code
B. Handles async tasks
C. Stops execution
D. Deletes memory

Answer: B

Explanation:
Event loop manages async execution.
Moves tasks from queue to stack.
Ensures non-blocking behavior.
Core concept in JS runtime.

Q23. Microtasks example

A. setTimeout
B. Promise.then
C. setInterval
D. DOM

Answer: B

Explanation:
Microtasks have higher priority.
Promises go into microtask queue.
Executed before macrotasks.
Faster execution order.

Q24. Promise represents

A. Immediate value
B. Future value
C. Null
D. Undefined

Answer: B

Explanation:
Promise handles async results.
It represents future completion.
Can be success or failure.
Used for async operations.

Q25. Callback hell problem

A. Fast execution
B. Deep nesting
C. Memory leak
D. Syntax error

Answer: B

Explanation:
Callback hell creates nested structure.
Hard to read and maintain.
Called Pyramid of Doom.
Solved using promises.

Q26. async keyword

A. stops execution
B. returns Promise
C. loops code
D. blocks thread

Answer: B

Explanation:
Async functions always return Promise.
Allows use of await.
Simplifies async code.
Introduced in ES6.

Q27. fetch API returns

A. string
B. object
C. Promise
D. number

Answer: C

Explanation:
Fetch is asynchronous.
Returns a Promise.
Resolved with response object.
Used for HTTP calls.

Q28. API error handling

A. ignore error
B. try-catch
C. loop
D. break

Answer: B

Explanation:
Errors handled using try-catch.
Also using .catch().
Important for stability.
Handles network and HTTP errors.

Q29. Debouncing use case

A. Sorting
B. Search input
C. Looping
D. Memory

Answer: B

Explanation:
Debouncing prevents frequent calls.
Used in search bars.
Improves performance.
Reduces API load.

Q30. Synchronous code

A. Parallel
B. Sequential
C. Random
D. Async

Answer: B

Aptitude MCQ

Q1.

A shopkeeper buys an item for ₹100 and sells it for ₹120. Find profit %.
Options:
A. 10% | B. 15% | C. 20% | D. 25%
Answer: C

Solution (Step by Step):

1. Profit = SP − CP = 120 − 100 = 20
2. Profit % = (Profit ÷ CP) × 100 = (20 ÷ 100) × 100 = 20%

Q2.

CP = ₹200, Loss = ₹50. Find Loss %.
Options:
A. 20% | B. 25% | C. 30% | D. 35%
Answer: B

Solution:

1. Loss % = (Loss ÷ CP) × 100 = (50 ÷ 200) × 100 = 25%

Q3.

If SP = ₹150 and profit = 50%, find CP.
Options:
A. ₹100 | B. ₹120 | C. ₹90 | D. ₹110
Answer: A

Solution:

1. Profit % = 50% → SP = 150% of CP
2. CP = SP ÷ 1.5 = 150 ÷ 1.5 = ₹100

Q4.

An item is sold at ₹80 with 20% loss. Find CP.
Options:
A. ₹90 | B. ₹95 | C. ₹100 | D. ₹110
Answer: C

Solution:

1. Loss = 20% → SP = 80% of CP
2. CP = SP ÷ 0.8 = 80 ÷ 0.8 = ₹100

Q5.

Profit = ₹40, CP = ₹160. Find SP.
Options:
A. ₹180 | B. ₹200 | C. ₹210 | D. ₹190
Answer: B

Solution:

1. SP = CP + Profit = 160 + 40 = ₹200

Q6.

If profit % = 25% and CP = ₹400, find SP.
Options:
A. ₹450 | B. ₹500 | C. ₹480 | D. ₹520
Answer: B

Solution:

1. Profit = 25% of 400 = 100
2. SP = CP + Profit = 400 + 100 = 500

Q7.

A trader marks 20% above CP and gives 10% discount. Find profit %.
Options:
A. 8% | B. 10% | C. 12% | D. 15%
Answer: A

Solution:

1. Marked Price = 120% of CP
2. After 10% discount → SP = 90% of 120 = 108% of CP
3. Profit % = 108 − 100 = 8%

Q8.

CP = ₹250, SP = ₹300. Find profit.
Options:
A. ₹40 | B. ₹50 | C. ₹60 | D. ₹70
Answer: B

Solution:

1. Profit = SP − CP = 300 − 250 = ₹50

Q9.

Loss % = 10%, SP = ₹180. Find CP.
Options:
A. ₹190 | B. ₹200 | C. ₹210 | D. ₹220
Answer: B

Solution:

1. SP = 90% of CP → CP = SP ÷ 0.9 = 180 ÷ 0.9 = ₹200

Q10.

Profit % = 50%, CP = ₹300. Find SP.
Options:
A. ₹400 | B. ₹450 | C. ₹500 | D. ₹550
Answer: B

Solution:

1. Profit = 50% of 300 = 150
2. SP = CP + Profit = 300 + 150 = 450

Q11.

SI on ₹1000 at 10% for 2 years.
Options:

A. ₹100 | B. ₹200 | C. ₹300 | D. ₹400
Answer: B

Solution:

1. SI = (P × R × T)/100 = (1000 × 10 × 2)/100 = 200

Q12.

Find total amount (SI) for above.
Options:
A. ₹1100 | B. ₹1200 | C. ₹1300 | D. ₹1400
Answer: B

Solution:

1. Amount = P + SI = 1000 + 200 = 1200

Q13.

CI on ₹1000 at 10% for 2 years.
Options:
A. ₹200 | B. ₹210 | C. ₹220 | D. ₹230
Answer: B

Solution:

1. Year 1 → 1000 × 1.1 = 1100
2. Year 2 → 1100 × 1.1 = 1210
3. CI = 1210 − 1000 = 210

Q14.

Difference between CI & SI (above case).
Options:
A. ₹5 | B. ₹10 | C. ₹15 | D. ₹20
Answer: B

Solution:

1. CI − SI = 210 − 200 = 10

Q15.

SI formula.
Options:
A. P+R+T | B. (P×R×T)/100 | C. P×T | D. R×T
Answer: B

Solution:

1. SI = (P × R × T)/100

Q16.

Rate = 5%, P = 2000, T = 3. Find SI.
Options:
A. ₹200 | B. ₹250 | C. ₹300 | D. ₹350
Answer: C

Solution:

1. SI = (2000 × 5 × 3)/100 = 300

Q17.

Amount after 1 year (CI) on ₹1000 at 10%.
Options:
A. ₹1100 | B. ₹1200 | C. ₹1050 | D. ₹1000
Answer: A

Solution:

1. CI = 1000 × 10/100 = 100
2. Amount = 1000 + 100 = 1100

Q18.

If CI = SI, then time is
Options:
A. 1 year | B. 2 years | C. 3 years | D. 4 years
Answer: A

Solution:

1. CI = SI only in the first year (no compounding effect yet)

Q19.

Principal = ₹500, SI = ₹50, Rate = 10%. Find time.
Options:
A. 1 year | B. 2 years | C. 3 years | D. 4 years
Answer: A

Solution:

1. SI = (P × R × T)/100 → 50 = (500 × 10 × T)/100
2. 50 = 50 × T → T = 1 year

Q20.

CI grows faster than SI because
Options:
A. Higher rate | B. Compounding | C. Less time | D. Fixed rate
Answer: B

Solution:

1. CI adds interest on interest; SI only on principal

Q21.

Speed formula.
Options:
A. D × T | B. D ÷ T | C. T ÷ D | D. D + T
Answer: B

Solution:

1. Speed = Distance ÷ Time

Q22.

Distance = 100 km, Time = 2 hr. Speed?
Options:
A. 40 | B. 50 | C. 60 | D. 70
Answer: B

Solution:

1. Speed = 100 ÷ 2 = 50 km/hr

Q23.

Time = Distance ÷ Speed?
Options:
A. True | B. False | C. Depends | D. None
Answer: A

Solution:

1. Time = Distance ÷ Speed

Q24.

If speed doubles, time becomes?
Options:
A. Double | B. Half | C. Same | D. Zero
Answer: B

Solution:

1. Time ∝ 1/Speed → If speed doubles, time halves

Q25.

60 km/hr = ? m/s
Options:
A. 16.67 | B. 20 | C. 25 | D. 15
Answer: A

Solution:

1. Multiply by 5/18 → 60 × 5 ÷ 18 = 16.67 m/s

Q26.

10 m/s = ? km/hr
Options:
A. 18 | B. 36 | C. 20 | D. 25
Answer: B

Solution:

1. Multiply by 18/5 → 10 × 18 ÷ 5 = 36 km/hr

Q27.

Distance covered in 3 hrs at 40 km/hr?
Options:
A. 100 | B. 120 | C. 140 | D. 150
Answer: B

Solution:

1. Distance = Speed × Time = 40 × 3 = 120 km

Q28.

A train travels 300 km in 5 hrs. Speed?
Options:
A. 50 | B. 60 | C. 70 | D. 80
Answer: B

Solution:

1. Speed = 300 ÷ 5 = 60 km/hr

Q29.

If distance fixed, speed ↑ then time ↓
Options:
A. True | B. False | C. Depends | D. None
Answer: A

Solution:

1. Time ∝ 1/Speed → Increasing speed reduces time

Q30.

Formula for distance
Options:
A. S ÷ T | B. S × T | C. T ÷ S | D. S + T
Answer: B

Solution:

1. Distance = Speed × Time

Program for Practice

Q1. Change Text Content

Problem: Write a program to change the text of an element with id "title" to "Hello World!" after button click

Q2. Change Background Color

Problem: Change the background color of all paragraphs (<p>) to yellow means create 5 <p> tag on html and change background color of all after button click

Q3. Add a New Element

Problem: Add a new <li> element with text "New Item" to a <ul> with id "list". after button click
Description: Use createElement and appendChild.

Q4. Remove an Element

Problem: Remove the element with class "remove-me". after button click
Description: Practice querySelector and .remove().

Q5. Toggle Class

Problem: Toggle class "active" on a button with id "btn" when clicked. means you have to change color of button red when click first time and second time green and vice versa

Q6. Input Value Display

Problem: Display the value of an input with id "name" in a paragraph with id "output" when a button is clicked.

Q7. Count Elements

Problem: Count how many elements exist on the page and display in console.
means when we click button and if we have 10 tags in html page then show the all tag count present on html page

Q8. Change Image Source

Problem: Change the src attribute of an image with id "pic" to "new.jpg".
means you have to change the image after button click

Q9. Style Multiple Elements

Problem: Set font size of all elements with class "text" to 20px.
means you have to create one text box and one button and when user click on button then you have to set font size 20px

Q10. Add Event to Multiple Buttons

Problem: Add click event to all buttons with class "click-btn" that alerts "Clicked!".
means you have to create 5 button and apply runtime click on every button

---

🔹 ARRAY METHOD PRACTICE (10 Questions)

Q1. Sum of Array

Problem: Find sum of numbers in array [1, 2, 3, 4, 5] using .reduce().

Q2. Filter Even Numbers

Problem: Filter even numbers from [1,2,3,4,5,6] using .filter().
Example:

Q3. Map to Square

Problem: Square each number in [1,2,3,4] using .map().
Example:

Q4. Find a Number

Problem: Find first number greater than 10 in [5,12,8,20] using .find().
Example:

Q5. Check All Positive

Problem: Check if all numbers in [1,2,3] are positive using .every().

Q6. Check Some Negative

Problem: Check if some numbers in [1,-2,3] are negative using .some().

Q7. Sort Numbers

Problem: Sort [5,2,9,1] in ascending order using .sort().

Q8. Join Array

Problem: Join ['a','b','c'] into a string with - separator using .join().

Q9. Add Elements

Problem: Add 6,7 to array [1,2,3,4,5] using .push().

Q10. Remove First Element

Problem: Remove first element of [10,20,30] using .shift().

⟐ ASYNCHRONOUS JS PRACTICE (10 Questions)

Q1. Simple Promise

Problem: Create a promise that resolves with "Done" after 2 seconds.

Q2. Async Function

Problem: Create async function returning "Hello Async" and log result.
Q3. Await Example

Q4. Promise Reject

Problem: Create a promise that rejects if number < 0, resolves if >=0.

Q5. Chained Promises

Problem: Multiply 2 → Add 3 → Log result using promises.

Q6. Async/Await + Fetch

Problem: Fetch https://jsonplaceholder.typicode.com/todos/1 and log title.

Q7. Callback Example

Problem: Create function sum(a,b,callback) that calls callback with sum.

Q8. Callback with Timeout

Problem: Call a callback after 2 seconds.

Q9. Promise.all

Problem: Wait for 2 promises and log results.

Q10. Error Handling

Problem: Async function throws error, catch it.